

RISC-V Processor Project - Pipeline

Team Name: RISK_3

Team No: 28

Team Members:

1. Nagamalla Sai Abhinav – Roll No: 2024102037
2. Nikhil Venkat Atkuru – Roll No: 2024102039
3. Abhinav Venkata Kota – Roll No: 2024102059

1 Introduction

This report presents the design and implementation of a 5-stage pipelined RISC-V processor, consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. The primary objective is to achieve maximum throughput, while handling data hazards, control hazards, and structural hazards through hazard detection units, forwarding paths, and pipeline control mechanisms. Key optimizations include resolving branches in the ID stage to reduce flush penalties, multiple forwarding units for data hazard resolution, and an extra forwarding unit for load-store dependencies. These modifications demonstrate the trade-offs between hardware complexity and performance in efficient pipeline design.

2 Modifications to the Proposed Design (Bonus Part)

The design mentioned in the TB solves **beq** at EX stage. This will result in flushing 2 instructions if the branch is taken. So, we optimized this by resolving it in an earlier stage i.e. at ID stage using some additional hardware discussed later which will reduce the branch misprediction penalty. This addition is slower for a set of cases which introduces stalls compared to the TB design (again discussed in detail later in the report), but is better in most of the general cases. So, it is a trade off and we feel our's is a better design because flushes are wasted and cannot be recovered while stalls can be prevented by compilers.

We also recognized an extra forwarding case where a **ld** is followed by a **sd** and the data to be stored is actually the data which is just loaded. So, instead of wasting clock cycles on stalls, we directly forward it using additional hardware called **extra forward unit**.

As done in the sequential part, we continued the same implementation of complete **R and I format** instructions in addition to the required one. We also exploited the minimal ISA implementation which gave a lot of don't care conditions which helped us find simple combinational logic for the **Control** and **ALU Control** blocks which are much more efficient in terms of power, area and speed compared to the if else statements synthesized by verilog.

3 Hazards

These are the different types of hazards one might encounter while designing a pipelined RISC-V Processor.

3.1 Structural Hazards

Structural hazards arise from resource conflicts when the hardware cannot support all overlapped instructions simultaneously. This happens when two or more instructions in the pipeline require access to the exact same hardware resource (like memory, ALU, or registers) during the same clock cycle.

We can face problems when multiple instructions requiring a common functional unit are executed simultaneously. Most designs include additional hardware components, such as an adder for calculating PC increment and branch offsets based on the instruction type, as well as a comparator to check for branch instructions (e.g., beq). These components reduce dependence on the ALU, but they can introduce stalls, which diminishes the efficiency and purpose of pipelining.

3.2 Data Hazards

Data hazards occur when instructions exhibit data dependence, meaning one instruction relies on the result of a previous instruction that has not yet completed execution and updated the system state (registers or memory).

Types of Data Hazards

- **Read After Write (RAW):** A true dependence. Instruction B tries to read a register before Instruction A writes to it. B gets the old, incorrect value.
- **Write After Read (WAR):** An anti-dependence. Instruction B tries to write to a register before Instruction A reads it. A gets the new, incorrect value (rare in simple, strictly ordered pipelines, but common in out-of-order execution).
- **Write After Write (WAW):** An output dependence. Instruction B tries to write to a register before Instruction A writes to it. The register ends up containing A's value instead of B's.

Proposed Solutions: To avoid the above mentioned problems, we use Data Forwarding, pipeline stalls for (RAW) and compiler scheduling (software optimisation) do help them solve. Also, in our pipelining design, we will not face the problems for WAR and WAW hazards, because of the way how the pipeline is structured. WAW hazards are not a problem at all due to an unified **Write Back (WB)** stage and not writing directly after the ALU result is computed for arithmetic operations. WAR is not even an issue in this 5 stage pipeline because The write operation which follows read will not corrupt the data while the read operation takes place because the WB stage of write occurs much later after the ID stage of the read.

3.3 Control Hazards

Control hazards (also known as branching hazards) occur when the pipeline needs to make a decision based on a branch instruction (e.g., if-else conditions, loops) but the condition or the target address has not yet been calculated.

Because the pipeline fetches a new instruction every cycle, it does not know which instruction to fetch immediately after a branch until the branch condition is evaluated. If it fetches the wrong instructions, they must be discarded (flushed).

Causes

- Conditional branches (BEQ, BNE).
- Unconditional branches or jumps (JMP).
- Exceptions and interrupts.

Proposed Solutions To avoid the above mentioned problems, there are multiple solutions which can be implemented. Firstly, we can use branch prediction, in which the processor guesses the outcome of a branch (taken or not taken) before it is officially calculated. There are 2 main types of branch predictions: static and dynamic predictions. The static prediction uses fixed rules such as taken or not taken. Generally, we try to implement not taken, as it is more commonly seen (about 60% of the time). Dynamic prediction, which maintains a branch history Table, to track the behaviour in the past taken branches. The next solution is pipeline flushing, which is used when the prediction made by the processor is incorrect, then the pipeline flushes the incorrect instructions and restarts from the correct address. Lastly, we can also use delayed branching, which is a compiler/software based optimisation. We struck to simple solution of **Always Branch NOT TAKEN** and flush if the branch is taken.

4 Hazard Detection Unit

To avoid these errors, we add a Hazard Detection block.

The `hazard_detection` is a hardware component designed to monitor instructions currently moving through a processor's pipeline. Its primary function

is to assert a stall signal (set to 1) when it detects specific data hazards that cannot be resolved via standard data forwarding alone. By stalling, the pipeline freezes the Program Counter (PC) and the Instruction Fetch/Instruction Decode (IF/ID) pipeline registers, giving older instructions time to complete their memory reads or arithmetic operations before the dependent instruction proceeds.

4.1 Signal Analysis for Stalls

We have to analyze if a stall is required from the various control signals and register addresses:

Decode Stage (IF/ID):

- **rs1_IFID & rs2_IFID:** The source registers needed by the instruction currently decoding.
- **Branch_IFID:** Flag indicating if the decoding instruction is a branch.
- **MemWrite_IFID:** Flag indicating if the decoding instruction is a store (write to memory).

Execute Stage (ID/EX):

- **rd_IDEX:** The destination register of the instruction currently in the ALU.
- **MemRead_IDEX:** Flag indicating if the instruction in Execute is a Load.
- **RegWrite_IDEX:** Flag indicating if the instruction in Execute will write to a register.

Memory Stage (EX/MEM):

- **rd_EXMEM:** The destination register of the instruction currently accessing data memory.
- **MemRead_EXMEM:** Flag indicating if the instruction in Memory is a Load.

Output:

- **stall:** A 1-bit signal directing the pipeline to insert a bubble.

4.2 Hazard Detection Conditions

- **Load-Use Hazard:** We check if the instruction currently in the Execute stage is a Load (**MemRead_IDEX** is true). A stall occurs if the instruction directly behind it in the Decode stage needs to use the loaded value as its first operand (**rs1_IFID**).

- **Load-Use Hazard (Source Register 2) with Store Optimization:** Similar to the above case, but the dependency is on the second operand (`rs2_IFID`). If you are loading a value just to store it immediately into memory (e.g., `lw x5, 0(x10)` followed by `sw x5, 0(x11)`), the Store doesn't actually need the data in the Execute stage. It only needs the data in the Memory stage. Forwarding from the end of the Load's Memory stage to the beginning of the Store's Memory stage resolves this without a stall.
- **Branch Data Hazard (Execute):** We check if the instruction in Decode is a Branch, but it depends on the result of an arithmetic instruction currently in the Execute stage.
- **Branch Data Hazard (Memory):** We check if the instruction in Decode is a Branch, and it depends on a Load instruction that is currently in the Memory stage.

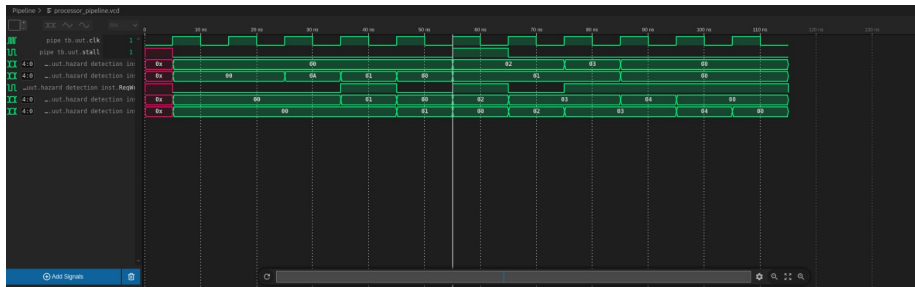


Figure 1: GTK-wave output of the Hazard detection unit which handles the arithmetic dependency on load

```
ld  x2, 0(x0)
add x3, x2, x1
```

The hazard detection unit identifies that the `add` instruction depends on the value loaded by the preceding `ld` instruction and inserts a stall until the loaded data becomes available.

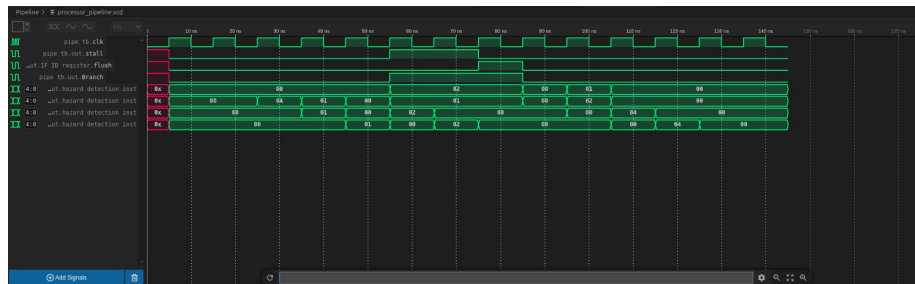


Figure 2: GTK-wave output of the Hazard Detection unit which handles the branch dependency on load

```
ld x2, 0(x0)
beq x2, x1, 8
```

Since the branch comparison occurs in the ID stage and the loaded value becomes available only after the MEM/WB stage, the hazard detection unit inserts two stall cycles before the branch instruction can proceed.

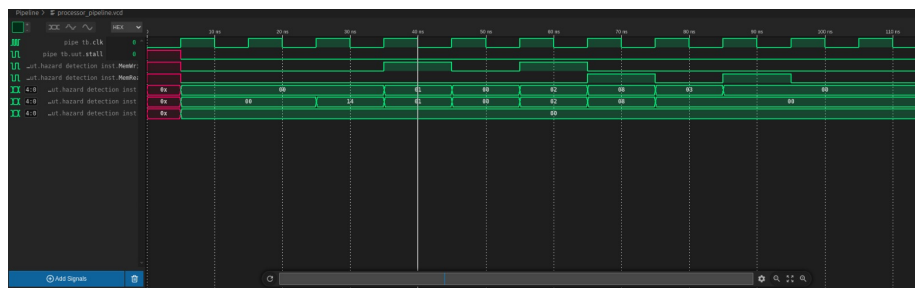


Figure 3: GTK-wave output of the Hazard Detection unit which handles store operation after load

```
ld x2, 8(x0)
sd x2, 8(x0)
```

The load instruction reads the value from memory after the store has completed its memory stage, so the correct value is obtained without requiring any pipeline stall.

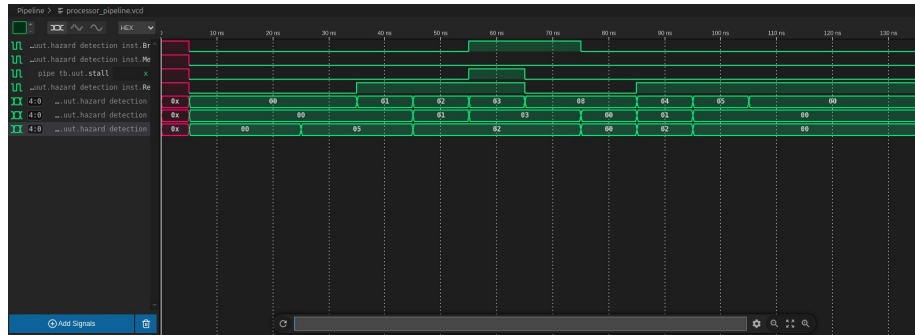


Figure 4: GTK-wave output of the Hazard Detection unit which handles branch dependency on arithmetic operations

```
add x3, x1, x2
beq x3, x2, 8
```

The branch instruction depends on the result of the preceding arithmetic instruction, so the hazard detection unit inserts a stall until the value of `x3` becomes available for branch comparison.

5 Pipeline Control Mechanisms: Stall and Flush

In an ideal pipeline, one instruction is fetched and one instruction completes every clock cycle, achieving an Instructions Per Cycle (IPC) of 1. However, physical data dependencies and control flow changes disrupt this ideal flow. To maintain architectural correctness without crashing the processor, the pipeline employs two primary control mechanisms: Stalling and Flushing.

5.1 The Stall Signal

What it is: A stall is a hardware mechanism that temporarily freezes the early stages of the pipeline while allowing the later stages to continue executing. It is effectively a controlled delay.

Why it is used: Stalls are required to resolve specific Data Hazards where data forwarding alone is insufficient. For instance, if a `load` instruction is fetching data from memory, that data does not physically exist on the processor's internal wires until the Memory (MEM) stage. If the immediately following instruction requires that data in the Execution (EX) or Decode (ID) stage, the pipeline must pause to wait for the memory read to complete.

How it works in our implementation: The `stall` signal is generated by the Hazard Detection Unit. When asserted (`stall = 1`), the hardware takes three simultaneous actions:

1. **Freezes the Program Counter (PC):** The PC's write-enable is disabled, forcing it to hold its current address so the same instruction is fetched again.
2. **Freezes the IF/ID Register:** The instruction currently in the Decode stage is trapped and held in place.
3. **Injects a Bubble (NOP):** All control signals passing from the ID stage to the ID/EX pipeline register are forced to zero. This creates a harmless "bubble" that propagates through the EX, MEM, and WB stages doing no work and modifying no state.

5.2 The Flush Signal

What it is: A flush is a hardware mechanism that forcibly clears the contents of a pipeline register, effectively deleting an instruction that was already fetched.

Why it is used: Flushes are required to resolve Control Hazards. Because the processor fetches a new instruction every clock cycle, it will blindly fetch the next sequential instruction while a branch (`beq`) is still being evaluated. If the branch evaluates to "Taken," the instruction that was just fetched into the pipeline is the wrong execution path and must be destroyed before it can modify any registers or memory.

How it works in our implementation: Because branch resolution was aggressively moved to the Decode (ID) stage, the flush penalty is minimized. When the branch comparators determine the condition is met, the `pc_ctrl` signal asserts. This signal is routed directly to the synchronous clear (or reset) pin of the IF/ID pipeline register, wiping the incorrectly fetched instruction and replacing it with a NOP. Simultaneously, the PC is updated with the branch target address.

5.3 Stall-Flush Interaction and Precedence Logic

A critical edge case arises when a branch instruction is subjected to a data hazard, requiring both signals to interact. Consider a branch in the ID stage waiting on a load instruction in the MEM stage. The Hazard Detection Unit asserts the `stall` signal. However, because the branch forwarding multiplexers might inadvertently route a stale memory address to the branch comparators during this wait period, the comparators might falsely evaluate the branch as "Taken" and assert the flush signal prematurely.

To prevent this "Self-Destructing Branch" anomaly, our implementation enforces strict hardware precedence: **Stalls must always override Flushes.**

This protection is implemented directly at the port mapping of the IF/ID register using a masking logic gate:

```
.flush(pc_ctrl & !stall)
```

This logic dictates that the flush signal is physically blocked from clearing the IF/ID register as long as the pipeline is stalled. The branch instruction is

safely held in the ID stage until the stall lifts, at which point the correct data is forwarded, the branch evaluates accurately, and the flush is permitted to execute if necessary.

6 IF-ID Pipeline Register

The IF/ID pipeline register is placed between the **Instruction Fetch (IF)** and **Instruction Decode (ID)** stage of the pipelined processor. The function of this pipeline register is to store the values sent on by the IF stage and then pass them onto the ID stage on the next clock cycle.

6.1 Input Signals

The IF/ID pipeline register receives the following input signals from PC

Signal Name	Width	Description
flush	1 bit	Control signal which say whether to flush the current values stored in the register or not.
stall	1 bit	Control signal saying whether to have a stall i.e. hold current values as it is
pc_in	64 bits	The address of the corresponding PC value, useful for calculating the branch target address
inst_in	32 bits	The current instruction which has all the useful information such as opcode, register addresses, etc

Table 1: Input signals of the IF/ID pipeline register

The output signals are also the same but just the names have out instead of in.

We have also implemented a synchronous reset which sets all the values of the pipeline register to 0 if the reset is asserted high.

7 ID-EX Pipeline Register

The ID/EX pipeline register is placed between the **Instruction Decode (ID)** and **Execute (EX)** stage of the pipelined processor. The function of this pipeline register is to store the values sent on by the ID stage and then pass them onto the EX stage on the next clock cycle.

7.1 Input Signals

The ID/EX pipeline register receives the following input signals from the Instruction Decode stage.

Signal Name	Width	Description
read	64 bits	Data read from the first source register in the register file.
read_data2_in	64 bits	Data read from the second source register in the register file.
imm_in	64 bits	Immediate value generated from the instruction.
funct_in	4 bits	Function field used by the ALU control unit to determine the operation.
rd_in	5 bits	Destination register identifier.
rs1_in	5 bits	First source register identifier.
rs2_in	5 bits	Second source register identifier.
ALUOp_in	2 bits	Control signal specifying the ALU operation type.
ALUSrc_in	1 bit	Selects the second ALU operand between register data and the immediate value.
MemRead_in	1 bit	Indicates whether a memory read operation is required.
MemtoReg_in	1 bit	Selects whether the data written back to the register file comes from memory or the ALU.
MemWrite_in	1 bit	Indicates whether a memory write operation is required.
RegWrite_in	1 bit	Enables writing the result back to the register file.

Table 2: Input signals of the ID/EX pipeline register

The output signals are also the same but just the names have out instead of in. This pipeline register is of very high importance as it holds all the control signals except branch. We also send the inputs of the register file in the ID stage that is rs1 and rs2 because we use them in checking conditions of the forwarding unit.

We have also implemented a synchronous reset which sets all the values of the pipeline register to 0 if the reset is turned high.

8 EX-MEM Pipeline Register

The EX/MEM pipeline register is placed between the **Execute (EX)** stage and the **Memory Access (MEM)** stage of the pipelined processor. The function of this pipeline register is to store the values generated by the EX stage and forward them to the MEM stage in the next clock cycle.

8.1 Input Signals

The EX/MEM pipeline register receives the following input signals from the Execute stage.

Signal Name	Width	Description
ALU_result	64 bits	The ALU's output which is either the result of the arithmetic operation or the load/store address
read_data2_in	64 bits	Data read from the second source register in the register file since this is the data which we are storing.
rd_in	5 bits	Destination register identifier, useful for the forwarding unit.
rs2_in	5 bits	Second source register identifier, useful for the load store forwarding unit described later.
MemRead_in	1 bit	Indicates whether a memory read operation is required.
MemtoReg_in	1 bit	Selects whether the data written back to the register file comes from memory or the ALU.
MemWrite_in	1 bit	Indicates whether a memory write operation is required.
RegWrite_in	1 bit	Enables writing the result back to the register file.

Table 3: Input signals of the EX/MEM pipeline register

The output signals are also the same but just the names have out instead of in.

We have also implemented a synchronous reset which sets all the values of the pipeline register to 0 if the reset is turned high.

9 MEM-WB Pipeline Register

The MEM/WB pipeline register is placed between the **Memory Access (MEM)** stage and the **Write Back (WB)** stage of the pipelined processor. The function of this pipeline register is to store the values generated by the MEM stage and forward them to the WB stage in the next clock cycle. These values are used to determine what data should be written back to the register file and which register should receive the result.

9.1 Input Signals

The MEM/WB pipeline register receives the following input signals from the Memory stage.

Signal Name	Width	Description
RegWrite_in	1 bit	Control signal indicating whether the result should be written back to the register file.
MemtoReg_in	1 bit	Selects whether the data written back to the register file comes from memory or from the ALU result.
MemRead_in	1 bit	Indicates whether a memory read operation was performed in the MEM stage.
rd_in	5 bits	Destination register identifier where the final result will be written.
ALU_result_in	64 bits	Result produced by the ALU during the Execute stage and forwarded through the MEM stage.
readdata_in	64 bits	Data read from memory during a load instruction.

Table 4: Input signals of the MEM/WB pipeline register

The output signals are the same just the suffix in is replaced with out. Although MemRead is not used in the WB stage we still send it into the MEM/WB pipeline register because like we are using this control signal in the extra forwarding unit. We have also implemented a synchronous reset which sets all the values of the pipeline register to 0 if the reset is turned high.

10 Forwarding Units

10.1 EX Stage Forwarding Unit

The Forwarding Unit is located in the Execution (EX) stage and is responsible for resolving standard Read-After-Write (RAW) data hazards. When an instruction in the EX stage requires data that has just been computed by an older instruction, waiting for that data to be written to and read from the Register File would introduce unnecessary stalls. This unit prevents those stalls by dynamically routing the computed data directly from the pipeline registers into the ALU multiplexers.

Hazard Detection and Priority Routing

The unit continuously compares the source registers (*rs1* and *rs2*) of the instruction currently in the EX stage against the destination registers (*rd*) of the older instructions currently residing in the MEM and WB stages. To handle sequences where multiple consecutive instructions write to the same register, it enforces strict priority logic:

- **EX/MEM Forwarding (Highest Priority):** If the instruction immediately preceding the current one (now in the MEM stage) writes to a matching source register, the unit asserts the multiplexer control signal (10) to forward the `ALU_ans` directly to the ALU input. This guarantees the ALU uses the most recently calculated data.

- **MEM/WB Forwarding:** If a matching destination register is detected in the WB stage (and not in the MEM stage), the unit asserts the control signal (01) to forward the `data_to_write` wire.
- **Default Path:** If no active data hazards are detected, the control signal remains 00, allowing the ALU to use the data originally read from the Register File during the ID stage.

Architectural Protections

The unit includes a hardwired protection mechanism (`rd != 5'b0`) to prevent the forwarding of the zero register (`x0`). This ensures that any software attempts to write to `x0` are physically ignored by the forwarding paths, maintaining the integrity of the RISC-V architecture.

Signal	Width	Description
rs1	5 bits	First source register of the instruction currently in the EX stage.
rs2	5 bits	Second source register of the instruction currently in the EX stage.
rd_EXMEM	5 bits	Destination register of the older instruction currently residing in the MEM stage.
rd_MEMWB	5 bits	Destination register of the oldest active instruction currently residing in the WB stage.
RegWrite_EXMEM	1 bit	Control signal indicating if the instruction in the MEM stage will write to the register file.
RegWrite_MEMWB	1 bit	Control signal indicating if the instruction in the WB stage will write to the register file.
fwdA	2 bits	Multiplexer control signal for <code>rs1</code> ALU input. 10 : Forward from MEM stage. 01 : Forward from WB stage. 00 : Use default Register File data.
fwdB	2 bits	Multiplexer control signal for <code>rs2</code> ALU input. 10 : Forward from MEM stage. 01 : Forward from WB stage. 00 : Use default Register File data.

Table 5: Inputs and Outputs of the EX-Stage Forwarding Unit

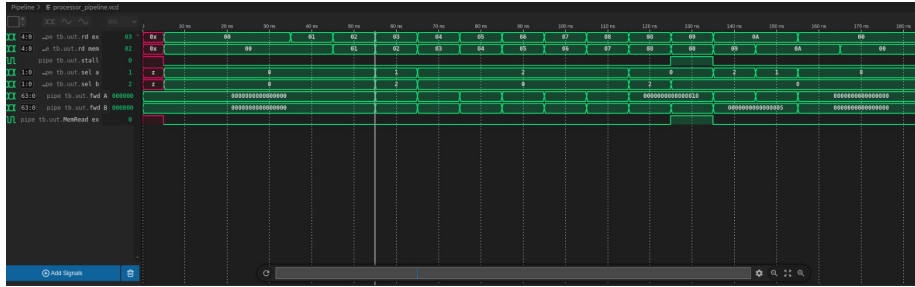


Figure 5: GTK-wave output of forwarding unit

```
add x3, x1, x2
add x4, x3, x1
```

Explanation: The forwarding unit detects when a source register matches the destination register of a previous instruction and forwards the value from the EX/MEM or MEM/WB pipeline stages to the ALU inputs. The forwarding unit detects when a source register matches the destination register of a previous instruction and forwards the value from the EX/MEM or MEM/WB pipeline stages to the ALU inputs.

10.2 Branch Forwarding Unit

The forwarding unit is located at the ID stage, which helps resolve branches one instruction earlier than the standard textbook design, thus reducing the number of cycles flushed when a branch is taken. The two values which **beq** uses for comparison might be updated by preceding instructions but not yet written to the register file, which would result in stale data. Instead of stalling the pipeline until the data is written back, the required data is dynamically forwarded.

Depending on how close the preceding data-modifying instructions are to the branch, this unit taps into the pipeline registers to route the most up-to-date values directly into the ID stage comparators:

- **MEM-to-ID Forwarding (Highest Priority):** If an older instruction currently in the MEM stage is writing to a destination register (*rd*) that matches the branch's *rs1* or *rs2*, the unit forwards the **ALU_result_mem** directly to the comparator.
- **WB-to-ID Forwarding:** If the instruction currently in the WB stage is writing to a matching register, the unit forwards the **data.to.write** wire directly to the comparator. This specific forwarding path is essential to bypass the internal write delay of the Register File. In this architecture, register writes typically resolve on the negative edge of the clock cycle. If the comparator were forced to wait for this write to physically complete before reading the updated value, the negative phase of the clock cycle would have to be significantly extended. Specifically, the time between

the falling edge and the next rising edge would need to accommodate the register read delay, the propagation delay of the branch comparator logic, and the setup time for the subsequent pipeline registers. This would severely degrade the maximum achievable clock frequency. Tapping directly into the `data_to_write` signal guarantees the branch comparator receives the updated value immediately at the beginning of the cycle, eliminating the need for an artificially elongated clock phase or an additional synchronization stall.

Critical Path Constraint and Stability Assumption

The aforementioned forwarding logic could theoretically be expanded to include an EX-to-ID forwarding path—routing the raw, combinational `ALU_result` directly from the ALU output into the branch comparators. However, this path was deliberately omitted.

Chaining the ALU computation logic directly into the branch comparison logic within a single clock cycle creates an excessively long critical path. This massive delay would dictate a much larger minimum clock period, forcing the entire processor to run at a significantly lower clock frequency. To maintain a high overall clock speed, the design operates under the architectural assumption that forwarding must only occur from stable pipeline registers. By waiting for the ALU output to latch securely into the EX/MEM pipeline register, it is guaranteed that the result is electrically stable before forwarding it back to the ID stage.

Performance Analysis: Textbook vs. Implemented Design

By moving branch resolution to the ID stage, the implemented design significantly outperforms the standard textbook baseline (which resolves branches in the EX stage) for normal taken branches. Because the branch decision is made one stage earlier, only one incorrect instruction (in the IF stage) needs to be flushed, saving a full clock cycle compared to the traditional two-cycle flush penalty.

However, the combination of early branch resolution and the strict omission of EX-to-ID forwarding alters the penalty distribution for branches dependent on immediately preceding instructions.

Scenario A: Arithmetic Dependency

```
add x1, x2, x3
beq x1, x4, target
```

In the **Textbook Implementation** (Branch in EX), the `beq` evaluates in the EX stage while the `add` is in the MEM stage. Forwarding handles this seamlessly with **0 stall cycles**.

- **Branch Not Taken:** 0 Stalls + 0 Flushes = **0 Cycle Penalty**
- **Branch Taken:** 0 Stalls + 2 Flushes = **2 Cycle Penalty**

In the **Implemented Design** (Branch in ID), the `beq` evaluates in the ID stage. Because EX-to-ID forwarding is prohibited, the pipeline must stall for **1 cycle** to let the `add` reach the MEM stage.

- **Branch Not Taken:** 1 Stall + 0 Flushes = **1 Cycle Penalty**
- **Branch Taken:** 1 Stall + 1 Flush = **2 Cycle Penalty**

Scenario B: Load Dependency

```
ld x1, 0(x2)
beq x1, x4, target
```

In the **Textbook Implementation**, the `beq` in EX must wait for the `ld` to finish its MEM phase, requiring **1 stall cycle**.

- **Branch Not Taken:** 1 Stall + 0 Flushes = **1 Cycle Penalty**
- **Branch Taken:** 1 Stall + 2 Flushes = **3 Cycle Penalty**

In the **Implemented Design**, the `beq` in ID must wait for the `ld` to reach the WB stage to be safely forwarded, requiring **2 stall cycles**.

- **Branch Not Taken:** 2 Stalls + 0 Flushes = **2 Cycle Penalty**
- **Branch Taken:** 2 Stalls + 1 Flush = **3 Cycle Penalty**

Conclusion

While the textbook design holds a slight 1-cycle advantage in the isolated cases of not-taken dependent branches, the implemented design is structurally superior on a macro level. Stall penalties caused by data dependencies (both the 1-stall arithmetic and 2-stall load cases) can be heavily mitigated by modern compilers through instruction scheduling—inserting independent instructions between the `add/ld` and the `beq`—or completely hidden by advanced architectures utilizing Out-of-Order Execution (OOE).

Conversely, the flush penalties associated with control hazards cannot be avoided by compiler scheduling and remain a fixed tax on performance unless a highly complex dynamic branch predictor is integrated. By trading stall cycles—which software can optimize away—for a permanent hardware-level reduction in unavoidable flushes, the implemented design achieves a higher overall Instructions Per Cycle (IPC) across standard, compiled workloads.

Signal	Width	Description
rs1	5 bits	First source register of the branch instruction currently in the ID stage.
rs2	5 bits	Second source register of the branch instruction currently in the ID stage.
rd_EXMEM	5 bits	Destination register of the older instruction currently residing in the MEM stage.
rd_MEMWB	5 bits	Destination register of the oldest active instruction currently residing in the WB stage.
RegWrite_EXMEM	1 bit	Control signal indicating if the instruction in the MEM stage will write to the register file.
RegWrite_MEMWB	1 bit	Control signal indicating if the instruction in the WB stage will write to the register file.
fwdA	2 bits	Multiplexer control signal for rs1 forwarding. 01 : Forward from MEM stage. 11 : Forward from WB stage. 00 : Use default Register File data.
fwdB	2 bits	Multiplexer control signal for rs2 forwarding. 01 : Forward from MEM stage. 11 : Forward from WB stage. 00 : Use default Register File data.

Table 6: Inputs and Outputs of the Branch Forwarding Unit

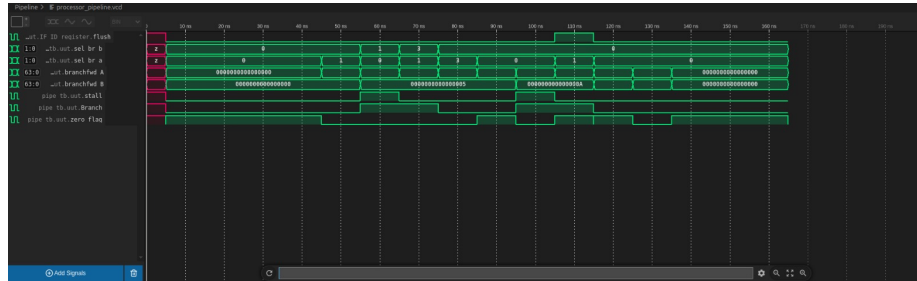


Figure 6: GTK-wave output of branch forwardig unit

```
beq x3, x2, 8
beq x5, x3, 8
```

These instructions demonstrate both cases of branch execution: the first branch is not taken since the registers are unequal, while the second branch is taken when the register values match. Hence we see no flush for the first branch and flush is done for the second branch.

10.3 MEM Forwarding unit

This forwarding unit is responsible for handling the load-store data hazard in the pipelined processor. This forwarding unit is used when the store instruction

uses the same register as that of the previous load instruction. If the location from which the data should be written on is coming from the same register to which the loaded value is being written to, it is forwarded. As the register file gets updated in the WB stage the store instruction would have to stall as it would have decoded the wrong value. To avoid stalling in such cases an extra forwarding unit is placed between the MEM and WB stage which handles this kind of hazard. This forwarding unit checks if the destination register of the load instruction matches with the source register of the store instruction. If such a condition occurs then the correct data is forwarded instead of stalling. There is a MUX at the output of this unit which decides whether the normal data should be sent or the forwarded data.

Signal	Width	Description
rd	5 bits	Destination register of the instruction producing the data .
rs	5 bits	Source register of the store instruction which contains the data to be written to memory.
MemRead	1 bit	Control signal indicating that the instruction is a load operation that reads data from memory.
MemWrite	1 bit	Control signal indicating that the instruction is a store operation that writes data to memory.
ef_mux_select	1 bit	Output control signal for the multiplexer. When set to 1, forwarded data is selected instead of register file data.

Table 7: Inputs and Output of Extra Forwarding Unit

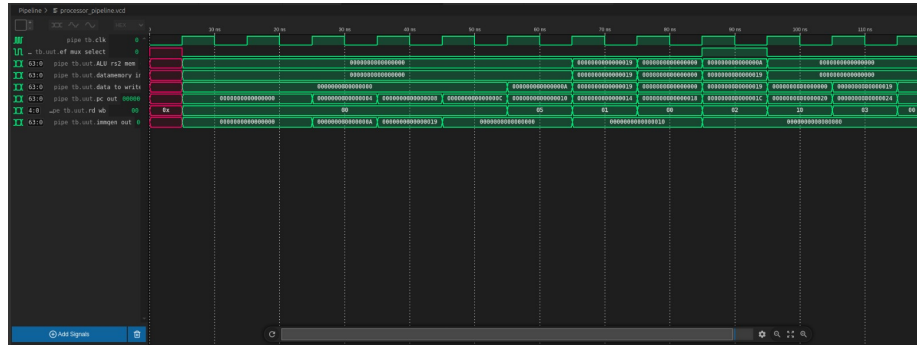


Figure 7: GTK-wave output for load store forwarding

```
ld x2, 0(x5)
sd x2, 16(x5)
```

For these type of instructions the load-store forwarding unit detects that the destination register of the load matches the source register of the store and

forwards the loaded data directly to the store instruction, avoiding a pipeline stall.

11 Muxes

Several multiplexers are used in the pipeline to select the correct data path based on control signals and hazard conditions.

11.1 PC Selection Multiplexer

The PC multiplexer selects the next value of the Program Counter. It chooses between the sequential PC value ($PC + 4$) produced by the PC Adder and the branch target address generated by the Branch Adder. The control signal `pc_ctrl` determines the selection. When the branch condition is satisfied, the branch target address is selected otherwise, the processor continues sequential execution.

11.2 ALU Operand Multiplexer

The ALU operand multiplexer selects the second input to the ALU. It chooses between the forwarded register value and the sign-extended immediate value generated by the `immgen`. The selection is controlled by the signal `ALUSrc`. For register-type instructions the register value is used, while for immediate or memory instructions the immediate value is selected.

11.3 Forwarding Multiplexer for Operand A

This multiplexer resolves data hazards for the first ALU operand. It selects between the value read from the register file, the ALU result from the EX/MEM stage, the write-back data from the MEM/WB stage, or a default value. The control signal generated by the forwarding unit determines which source contains the most recent value.

11.4 Forwarding Multiplexer for Operand B

This multiplexer performs forwarding for the second ALU operand. Similar to the first forwarding multiplexer, it selects between the register file output, forwarded ALU results from later pipeline stages, or write-back data. The forwarding unit generates the control signal to ensure the ALU receives the correct operand.

11.5 Branch Forwarding Multiplexer for Operand A

This multiplexer is used during branch comparison operations. It selects the correct value for the first operand of the branch comparison logic by choosing

between the register file output and forwarded values from later pipeline stages. This ensures that branch decisions are made using the most up-to-date data.

11.6 Branch Forwarding Multiplexer for Operand B

The second branch forwarding multiplexer provides the appropriate value for the second operand in branch comparisons. It forwards values from later pipeline stages when necessary, preventing incorrect branch decisions due to data hazards.

11.7 Write-Back Multiplexer

The write-back multiplexer selects the value that will be written back to the register file. It chooses between the ALU result and the data read from memory. The control signal `MemoReg` determines the selection. Arithmetic instructions write back the ALU result, while load instructions write back the memory data.

11.8 Load-Store Forwarding Multiplexer

The load-store forwarding multiplexer resolves hazards between load and store instructions. It selects the correct data that should be written to memory during a store operation. When the store instruction requires a value that was recently loaded but not yet written back to the register file, the extra forwarding unit enables this multiplexer to forward the value directly from the write-back stage.

11.9 ID/EX Stage Control Signal Multiplexer

The ID/EX stage control signal multiplexer is used to execute stall (NOP). The control signal to it is 'stall'. If it is asserted to high, all the control signals going into ID/EX pipeline are made 0 else the control block's outputs are only sent to the pipeline register. It is implemented as 5 1-bit wide 2*1 MUXs and a 2-bit wide 2*1 MUX for ALUOp.

12 Adders

12.1 PC Increment Adder

The PC Adder is responsible for the generation of the next instruction in the pipeline. This adder adds the current PC and the constant 4 so as to go the next instruction as the instructions are 4 bytes in RISC-V. The output thus contains the next sequential instruction which gets updated in the PC after the next clock cycle. This output is passed on to the PC multiplexer which selects this instruction if branch operation is not taken.

12.2 Branch Target Adder

The branch adder calculates the target address for branch instructions. This adder adds the current PC instruction with the immediate value obtained from the immgen module as this immediate value gives the branch offset to calculate the target address. The output of this branch target adder is then sent to the PC multiplexer where this output is selected if the branch is taken.

12.3 ID-Stage Branch Comparator

To resolve branches early without causing structural hazards with the main ALU in the EX stage, a dedicated comparator was implemented in the Decode (ID) stage. Instead of duplicating a resource-heavy ALU, this unit evaluates branch equality (**beq**) by performing a bitwise XOR on the forwarded source registers. The result is then fed into a zero detector which is just a NOR reduction tree. If the XOR output is all zeros, the branch condition is met and instantly asserted.

References

- [1] David A. Patterson and John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. Morgan Kaufmann.